Gmail Digest Agent

Architecture and build guide for a local, zero-cost morning email summary pushed to iPhone

Owner	Ankur Kar
Version	1.0 - built 11 October 2026
Host	iMac, Apple M4, 24 GB unified memory, macOS
Model	Google Gemma 4 12B QAT (Q4_0 GGUF), run locally in Bionic (LM Studio)
Schedule	Daily 8:00 AM IST via launchd; Mac wakes at 7:58 via pmset
Delivery	Telegram bot push notification
Running cost	Zero

This document records what the agent does, how its parts fit together, every setup step in order, the problems hit during the build and how each was fixed, and how to operate and extend it. The final source code is in the appendix.

1. What the agent does

Every morning the agent reads the last 24 hours of Gmail inbox mail, asks a language model running on the iMac to write a short digest, and pushes that digest to the iPhone. Actionable items come first. If any step fails, a failure alert is pushed instead, so a broken run is never silent.

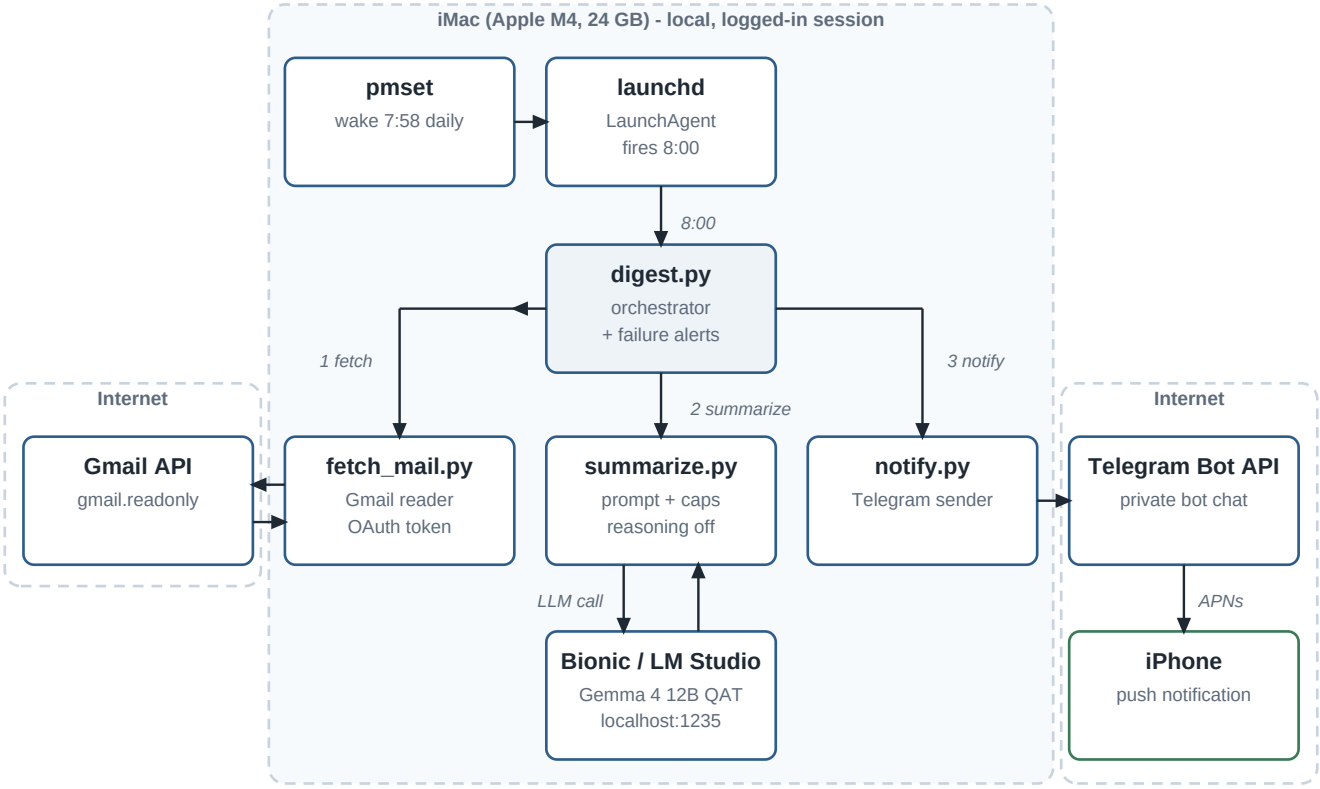
Design goals

- **Private:** email content is processed only on the iMac. Only the short summary leaves the machine.
- **Free:** no paid API, service or hardware. Gmail API, GitHub Pages, LM Studio and Telegram are all free.
- **Unattended:** wakes the Mac, starts the model server if needed, and recovers without user action.
- **Observable:** every run writes a log line; every failure sends a push alert.
- **Swappable:** model, endpoint and limits are configuration, so a cloud model can replace the local one later.

Agent loop in one line

Trigger (schedule) → **Perceive** (fetch email) → **Reason** (LLM summary) → **Act** (push notification) → **Report** (log and failure alert). This is the same shape every larger agent follows; later agents add tools and memory to the Reason step.

2. Architecture



Numbered arrows show the order of one run: 1 fetch mail, 2 summarize, 3 notify. Before step 1, digest.py checks the model server and opens Bionic if it is down. Gmail and Telegram are reached over HTTPS.

Components

Component	Role	Technology
pmset repeat	Wakes or powers on the Mac at 7:58 every day	macOS power management
LaunchAgent	Runs digest.py at 8:00; runs late after wake if the Mac slept	launchd, ~/Library/LaunchAgents/com.ankurkar.gmail-digest.plist
digest.py	Orchestrator: ensure server, fetch, summarize, notify, log, alert on failure	Python 3.13 in a project venv
fetch_mail.py	Signs in to Gmail and returns sender, subject, date and plain-text body per email	google-api-python-client, OAuth 2.0 desktop flow
summarize.py	Builds the prompt, caps its size, calls the model with reasoning disabled	openai Python client against an OpenAI-compatible local endpoint
Bionic (LM Studio)	Serves Gemma 4 12B QAT on localhost; loads the model just in time	llama.cpp GGUF runtime, port 1235
notify.py	Sends the digest or failure alert to the iPhone	Telegram Bot API over HTTPS
GitHub Pages site	Home page and privacy policy required to publish the OAuth app	ankur-kar7924.github.io

Key design decisions

Decision	Chosen	Why
Model location	Local (Gemma 4 12B QAT)	Emails stay on the iMac; free. 12B fits 24 GB with headroom, and QAT keeps quality close to full precision at 4-bit.

Decision	Chosen	Why
Model size	12B, not 27B	The 27B build is 16 GB and leaves too little memory for macOS and context on a 24 GB Mac.
Gmail access	Gmail API, read-only scope	Official, least privilege, reusable for later agents. The agent cannot send, delete or change mail.
OAuth app status	In production (unverified)	Testing mode expires refresh tokens every 7 days, which would silently break the daily job.
Push channel	Telegram bot	Free and reliable on iOS; two-way, enabling a conversational agent later. ntfy failed on iOS; Pushover costs money.
Scheduler	launchd, not cron	launchd runs a missed job when the Mac wakes; cron skips it.
Reasoning mode	Disabled	Gemma's thinking mode made a 3-email digest take 298 s; disabling it cut that to about 15 s.

3. What happens in one run

Time	Step	Detail
7:58	Wake	pmset wakes the iMac (or powers it on; see section 7 for the login caveat).
8:00	Trigger	launchd starts .venv/bin/python digest.py in the project folder.
+0 s	Ensure model server	GET /v1/models on port 1235. If it does not answer, run 'open -g -a Bionic' and poll every 5 s for up to 120 s.
+1 s	Fetch	Gmail query: in:inbox newer_than:1d -category:promotions -category:social, at most 50 messages, body truncated to 1,500 characters each. Token refreshes silently.
+2 s	Summarize	System prompt plus all emails (capped at 24,000 characters) sent to google/gemma-4-12b-qat at temperature 0.2 with reasoning_effort none. Model loads just in time.
+10-25 s	Notify	Header 'Morning digest - N emails' plus digest sent to the Telegram bot chat.
end	Log	One line appended to logs/digest.log: timestamp, OK, email count, duration.
on error	Alert	Error logged with traceback, failure alert pushed, process exits 1.

Measured performance

Scenario	Result
Reasoning on, 3 emails	298.1 s (about 2,500 hidden reasoning tokens)
Reasoning off, 3 emails, server already running	9-15 s
Bionic fully quit, cold start by the script	22.7 s
Run fired through launchd	16.8 s for 4 emails
Raw generation speed	About 9 tokens/s (memory-bandwidth bound on base M4)

4. Project layout and configuration

```
~/agents/gmail-digest/
  .venv/              Python virtual environment
  .env                secrets and settings (chmod 600)
  credentials.json    Google OAuth client (chmod 600)
  token.json          Gmail refresh token, created on first sign-in (chmod 600)
  fetch_mail.py       Gmail reader
  summarize.py        prompt and LLM call
  notify.py           Telegram sender
  digest.py           orchestrator (entry point)
  logs/digest.log     run history
~/Library/LaunchAgents/com.ankurkar.gmail-digest.plist   8:00 schedule
```

Setting	Where	Value / default
TELEGRAM_BOT_TOKEN	.env	secret from @BotFather
TELEGRAM_CHAT_ID	.env	your private chat with the bot
LLM_BASE_URL	.env (optional)	http://localhost:1235/v1
LLM_MODEL	.env (optional)	google/gemma-4-12b-qat
MAX_PROMPT_CHARS	.env (optional)	24000
LLM_APP_NAME	.env (optional)	Bionic
QUERY, MAX_EMAILS, MAX_BODY_CHARS	fetch_mail.py	see run flow above
Context length	Bionic, Local Model Defaults	at least 16,384 tokens

The .env file also holds an unused NTFY_TOPIC line from an earlier attempt; it can be deleted.

5. Build process, step by step

The order below is the order that worked. Each phase can be checked before the next starts.

A. Choose and run the model

- Installed Bionic (an LM Studio build) instead of Ollama; one local server is enough.
- Filtered Hugging Face by: text generation, fits memory (about 0.6 GB per billion parameters at 4-bit), GGUF or MLX format, official publisher, instruct variant.
- Downloaded Gemma 4 12B QAT, Q4_0 GGUF (only GGUF is published for QAT). Saved under ~/.lmstudio/models/lmstudio-community/gemma-4-12B-it-QAT-GGUF/.
- Started the Local Model API server (it came up on port 1235) and confirmed the model ID with curl /v1/models.
- Turned on just-in-time model loading; turned on log redaction and off verbose logs so email text is not written to disk.

B. Python project

- python3 --version: 3.13.0.
- Created ~/agents/gmail-digest with a .venv and installed openai, google-api-python-client, google-auth-oauthlib, google-auth-httplib2, python-dotenv, requests.

C. Google Cloud and Gmail OAuth

- Created project gmail-digest with the personal Gmail account and enabled the Gmail API.
- Google Auth Platform: app name, support email, audience External, developer contact.
- Publish was blocked until Branding had a home page and privacy policy, so built a GitHub Pages site (repo Ankur-Kar7924.github.io) with index.html and privacy.html.
- Added both URLs and the authorized domain ankur-kar7924.github.io, saved, and published the app to In production.
- Created a Desktop OAuth client, downloaded the JSON and saved it as credentials.json (chmod 600).
- First run of fetch_mail.py opened the browser; accepted the 'unverified app' warning via Advanced; token.json was written.

D. Summarizer

- Wrote summarize.py with a strict system prompt: plain text, ACTION items first, keep every amount, date and sender, at most 15 words per line, under 600 characters, never invent details.
- Disabled reasoning with extra_body reasoning_effort none, which LM Studio honours.

E. Push notifications

- Tried ntfy: messages reached the app but iOS never showed a push (a known ntfy iOS limitation).
- Switched to a Telegram bot from @BotFather. Stored the token with a hidden 'read -s' prompt; read the chat ID from getUpdates after sending the bot a message; tested sendMessage.

F. Orchestrator and self-recovery

- Wrote notify.py and digest.py with failure alerts.
- The bundled lms CLI could not start the server on its own, but opening the app could, so digest.py opens Bionic hidden when the server is down. Verified by quitting Bionic and running the job.

G. Scheduling

- Created the LaunchAgent plist, validated it with plutil, loaded it with launchctl bootstrap, and fired it once with launchctl kickstart.
- Set sudo pmset repeat wakeorpoweron MTWRFSU 07:58:00 so the Mac is awake at 8:00.

6. Operating the agent

To	Command
See run history	cat ~/agents/gmail-digest/logs/digest.log
Run now (as launchd would)	launchctl kickstart -k gui/$(id -u)/com.ankurkar.gmail-digest
Run by hand	cd ~/agents/gmail-digest && source .venv/bin/activate && python digest.py
Preview a digest without sending	python summarize.py
Pause the schedule	launchctl bootout gui/$(id -u)/com.ankurkar.gmail-digest
Resume the schedule	launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.ankurkar.gmail-digest.plist
Check the wake schedule	pmset -g sched
Remove the wake schedule	sudo pmset repeat cancel
Check the model server	curl http://localhost:1235/v1/models
Revoke Gmail access	myaccount.google.com/permissions, then delete token.json

Mac state at 8:00

State	Outcome
Awake or screen locked	Runs on time (a locked screen is still a logged-in session)
Asleep	Woken at 7:58 by pmset, runs at 8:00
Shut down	pmset powers it on, but it stops at the login screen; the job runs only after you log in
Logged out	Does not run; LaunchAgents need a logged-in user

Leave the iMac asleep, not shut down, overnight.

7. Problems hit during the build and their fixes

Symptom	Cause	Fix
Digest took 298 s for 3 emails	Gemma thinking mode generated about 2,500 hidden tokens	extra_body={"reasoning_effort": "none"}; now about 15 s
Test summary dropped the bill amount	Prompt only said 'summarize in one line'	System prompt requires keeping amounts, dates and senders
Publish app greyed out	External production apps need home page and privacy policy	GitHub Pages site plus authorized domain
'Invalid domain: cannot contain whitespace'	Leading space copied with the domain	Cleared the field and typed the domain by hand

Symptom	Cause	Fix
FileNotFoundError: credentials.json	Downloaded file moved but not renamed	mv client_secret_*.json credentials.json
ntfy messages visible in app, no push	ntfy iOS app delivery is unreliable	Replaced with a Telegram bot
.env missing	Generate command was not run	Re-ran it; verify with ls -la
Telegram getUpdates returned no messages	No message had reached that bot yet	Confirmed bot username with getMe, sent 'hi' to it, re-ran
lms not found	App is named Bionic.app; CLI is bundled inside it	find /Applications/Bionic.app -name lms
lms server start failed	This build's CLI cannot start the daemon without the app	Script opens the app with 'open -g -a Bionic' instead
Server on port 1235, not 1234	Default port was taken	Endpoint set to 1235

Failure alert quick guide

Alert mentions	Likely cause and action
did not start within 120s	Bionic failed to launch or the model would not load; open it manually and check memory
RefreshError / invalid_grant	Gmail token revoked or password changed; delete token.json and run digest.py by hand to sign in again
ConnectionError to googleapis.com	No network at 8:00; check Wi-Fi reconnects after wake
context length / too many tokens	Very busy day; lower MAX_PROMPT_CHARS or raise context length
No alert and no digest	Mac was logged out or off, or the Telegram call itself failed; check logs/digest.log

8. Security and privacy

- **Least privilege:** the only Google scope is gmail.readonly.
- **Data flow:** full email text goes only to the local model on the iMac. The summary goes to Telegram, whose bot chats are not end-to-end encrypted.
- **Secrets:** credentials.json, token.json and .env are chmod 600 and must never be committed. Add them to .gitignore before putting the project in Git.
- **Logs:** LM Studio request redaction is on and verbose logging off, so email content is not written to its logs. digest.log holds only status lines.
- **Revocation:** remove access at myaccount.google.com/permissions; revoke the bot token with /revoke in @BotFather.
- **OAuth app:** 'In production' does not make it public; it is usable only with your credentials.json, and unverified apps are capped at 100 users.

9. Possible next steps

- **Prompt tuning:** adjust SYSTEM_PROMPT in summarize.py after a week of real digests.
- **Two-way bot:** poll Telegram for replies such as 'more on #2' and answer from the same emails. This turns the pipeline into a conversational agent with tools.
- **Deduplication and memory:** store processed message IDs in SQLite to avoid repeats and track open action items.

- **Job-alert pipeline:** reuse fetch_mail.py with a query for LinkedIn and Naukri alert emails, score each job against a profile file, and push a ranked shortlist.
- **Model swap:** point LLM_BASE_URL and LLM_MODEL at another local or cloud model without code changes.

Appendix: final source code

These are the versions running on the iMac. In digest.py the two emoji in the message strings (a sunrise in the header and a warning sign in the failure alert) are shown as [sunrise] and [warning] because the PDF font cannot draw emoji.

fetch_mail.py

```
import base64
import os
from pathlib import Path

from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

BASE = Path(__file__).resolve().parent
SCOPES = ["https://www.googleapis.com/auth/gmail.readonly"]
CREDS_FILE = BASE / "credentials.json"
TOKEN_FILE = BASE / "token.json"
QUERY = "in:inbox newer_than:1d -category:promotions -category:social"
MAX_EMAILS = 50
MAX_BODY_CHARS = 1500


def get_service():
    creds = None
    if TOKEN_FILE.exists():
        creds = Credentials.from_authorized_user_file(str(TOKEN_FILE), SCOPES)
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())
        else:
            flow = InstalledAppFlow.from_client_secrets_file(str(CREDS_FILE), SCOPES)
            creds = flow.run_local_server(port=0)
        TOKEN_FILE.write_text(creds.to_json())
        os.chmod(TOKEN_FILE, 0o600)
    return build("gmail", "v1", credentials=creds, cache_discovery=False)


def _decode(data):
    return base64.urlsafe_b64decode(data.encode()).decode("utf-8", errors="replace")


def _extract_text(payload):
    if payload.get("mimeType") == "text/plain" and payload.get("body", {}).get("data"):
        return _decode(payload["body"]["data"])
    for part in payload.get("parts", []) or []:
        text = _extract_text(part)
        if text:
            return text
    return ""


def fetch_recent_emails():
    service = get_service()
    resp = service.users().messages().list(
        userId="me", q=QUERY, maxResults=MAX_EMAILS
    ).execute()
    emails = []
    for ref in resp.get("messages", []):
        msg = service.users().messages().get(
            userId="me", id=ref["id"], format="full"
        ).execute()
        headers = {h["name"].lower(): h["value"] for h in msg["payload"].get("headers", [])}
        body = _extract_text(msg["payload"]).strip() or msg.get("snippet", "")
        emails.append({
            "from": headers.get("from", ""),
            "subject": headers.get("subject", "(no subject)"),
            "date": headers.get("date", ""),
            "body": " ".join(body.split())[:MAX_BODY_CHARS],
        })
    return emails


if __name__ == "__main__":
```

```
    emails = fetch_recent_emails()
    print(f"Fetched {len(emails)} emails\n")
    for e in emails:
        print(f"- {e['from']} | {e['subject']}")
```

summarize.py

```
import os
import time

from dotenv import load_dotenv
from openai import OpenAI

from fetch_mail import fetch_recent_emails

load_dotenv()
LLM_BASE_URL = os.getenv("LLM_BASE_URL", "http://localhost:1235/v1")
LLM_MODEL = os.getenv("LLM_MODEL", "google/gemma-4-12b-qat")
MAX_PROMPT_CHARS = int(os.getenv("MAX_PROMPT_CHARS", "24000"))

SYSTEM_PROMPT = """You write a short morning email digest that will be shown as a phone notification.
Rules:
- Plain text only. No markdown, no asterisks, no bullet symbols.
- Put anything that needs action first, each line starting with "ACTION:".
- Keep every amount, date, deadline and sender name exactly as written.
- One line per email worth mentioning, at most 15 words per line.
- Group or skip newsletters, marketing and routine automated notices unless they need action.
- Never invent details that are not in the emails.
- Keep the whole digest under 600 characters.
- If nothing needs attention, say so in one line."""


def build_user_prompt(emails):
    blocks = []
    total = 0
    for i, e in enumerate(emails, 1):
        block = f"Email {i}\nFrom: {e['from']}\nSubject: {e['subject']}\nBody: {e['body']}"
        if total + len(block) > MAX_PROMPT_CHARS:
            blocks.append(f"({len(emails) - i + 1} more emails omitted for length)")
            break
        blocks.append(block)
        total += len(block)
    return f"Here are {len(emails)} emails from the last 24 hours:\n\n" + "\n\n".join(blocks)


def summarize(emails):
    if not emails:
        return "No new emails in the last 24 hours."
    client = OpenAI(base_url=LLM_BASE_URL, api_key="lm-studio")
    resp = client.chat.completions.create(
        model=LLM_MODEL,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": build_user_prompt(emails)},
        ],
        temperature=0.2,
        extra_body={"reasoning_effort": "none"},
    )
    return resp.choices[0].message.content.strip()


if __name__ == "__main__":
    start = time.time()
    emails = fetch_recent_emails()
    digest = summarize(emails)
    print(f"--- {len(emails)} emails, {time.time() - start:.1f}s ---\n")
    print(digest)
```

notify.py

```
import os
from pathlib import Path

import requests
from dotenv import load_dotenv

load_dotenv(Path(__file__).resolve().parent / ".env")
```

```
def send_telegram(text):
    token = os.environ["TELEGRAM_BOT_TOKEN"]
    chat_id = os.environ["TELEGRAM_CHAT_ID"]
    resp = requests.post(
        f"https://api.telegram.org/bot{token}/sendMessage",
        data={"chat_id": chat_id, "text": text[:4000]},
        timeout=30,
    )
    resp.raise_for_status()
```

digest.py

```
import os
import subprocess
import sys
import time
import traceback
from datetime import datetime

import requests

from fetch_mail import fetch_recent_emails
from notify import send_telegram
from summarize import LLM_BASE_URL, summarize

LLM_APP_NAME = os.getenv("LLM_APP_NAME", "Bionic")
LLM_STARTUP_TIMEOUT = 120


def llm_server_up():
    try:
        return requests.get(f"{LLM_BASE_URL}/models", timeout=5).status_code == 200
    except requests.RequestException:
        return False


def ensure_llm_server():
    if llm_server_up():
        return
    print(f"LLM server not responding; opening {LLM_APP_NAME} in background")
    subprocess.run(["/usr/bin/open", "-g", "-a", LLM_APP_NAME], check=True)
    deadline = time.time() + LLM_STARTUP_TIMEOUT
    while time.time() < deadline:
        if llm_server_up():
            return
        time.sleep(5)
    raise RuntimeError(f"{LLM_APP_NAME} server did not start within {LLM_STARTUP_TIMEOUT}s")


def main():
    start = time.time()
    try:
        ensure_llm_server()
        emails = fetch_recent_emails()
        digest = summarize(emails)
        count = len(emails)
        header = f"[sunrise] Morning digest - {count} email{'' if count == 1 else 's'}"
        send_telegram(f"{header}\n\n{digest}")
        print(f"{datetime.now():%Y-%m-%d %H:%M} OK: {count} emails in {time.time() - start:.1f}s")
    except Exception as e:
        print(f"{datetime.now():%Y-%m-%d %H:%M} FAILED: {e}", file=sys.stderr)
        traceback.print_exc()
        try:
            send_telegram(f"[warning] Gmail digest failed: {type(e).__name__}: {str(e)[:300]}")
        except Exception:
            pass
        sys.exit(1)


if __name__ == "__main__":
    main()
```

com.ankurkar.gmail-digest.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
```

```
<dict>
    <key>Label</key>
    <string>com.ankurkar.gmail-digest</string>
    <key>ProgramArguments</key>
    <array>
        <string>/Users/ankurkar/agents/gmail-digest/.venv/bin/python</string>
        <string>/Users/ankurkar/agents/gmail-digest/digest.py</string>
    </array>
    <key>WorkingDirectory</key>
    <string>/Users/ankurkar/agents/gmail-digest</string>
    <key>StartCalendarInterval</key>
    <dict>
        <key>Hour</key>
        <integer>8</integer>
        <key>Minute</key>
        <integer>0</integer>
    </dict>
    <key>StandardOutPath</key>
    <string>/Users/ankurkar/agents/gmail-digest/logs/digest.log</string>
    <key>StandardErrorPath</key>
    <string>/Users/ankurkar/agents/gmail-digest/logs/digest.log</string>
    <key>RunAtLoad</key>
    <false/>
</dict>
</plist>
```